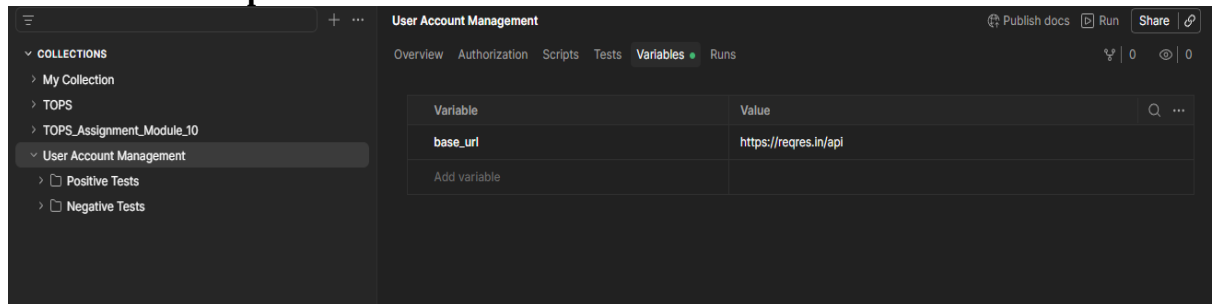


Section A: Conceptual Questions

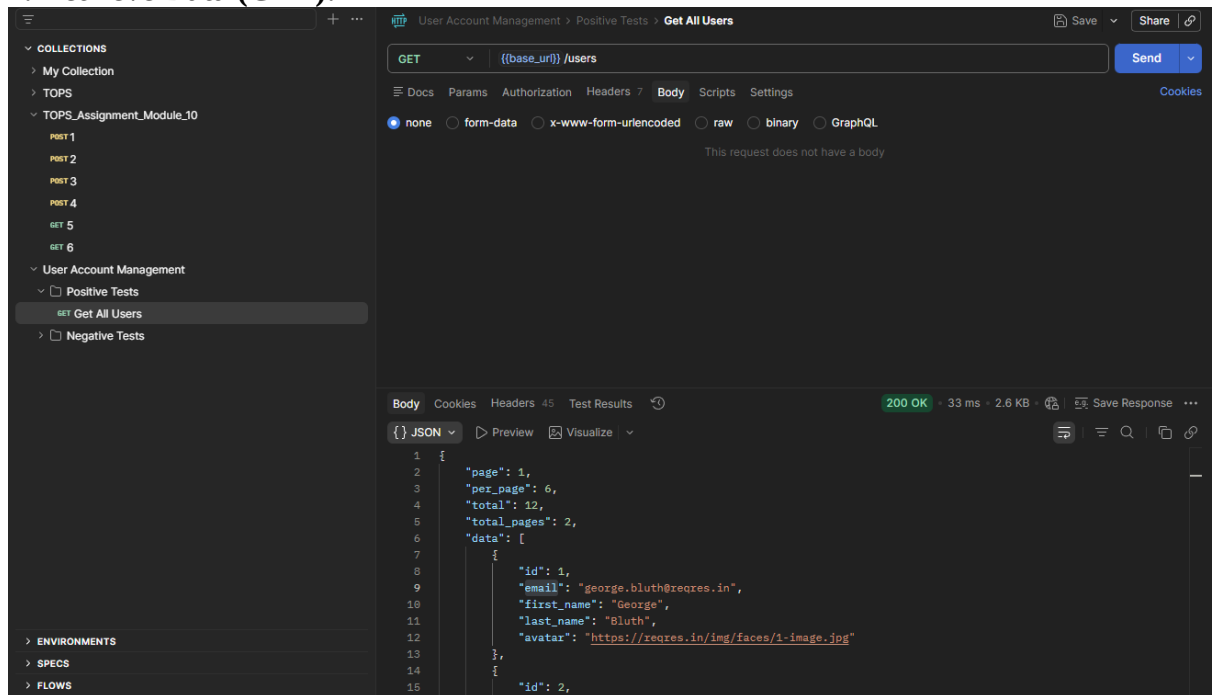
- 1. API testing allows us to test the core business logic directly without relying on a user interface. It is faster, more reliable, and less prone to breaking from minor visual changes.
- 2. This enables "shift-left" testing. Finding and fixing backend bugs early in the development cycle is significantly cheaper and faster than waiting for the full application to be built.
- 3. A 200 OK status only means the server received the request, but returning incorrect data means the test is a failure. You must always manually validate the response body payload against our expected results.
- 4. Headers provide the server with essential metadata, such as content type or authentication tokens. The request body contains the actual data payload like JSON that tells the server what information to process or store.
- 5. If we get an unauthorized response, manually verify that we have included the correct authentication credentials like a Bearer token or API key in our request headers and that the token has not expired.

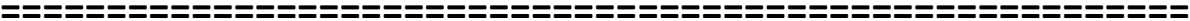
Section B: Practical Application Tasks

- 1. Collection Setup:

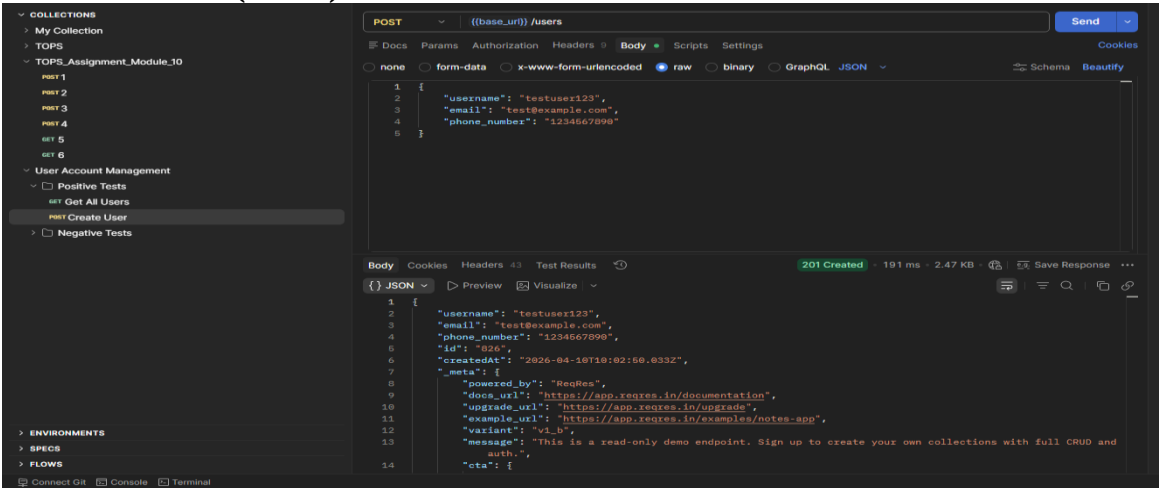


- 2. Retrieve Data (GET):

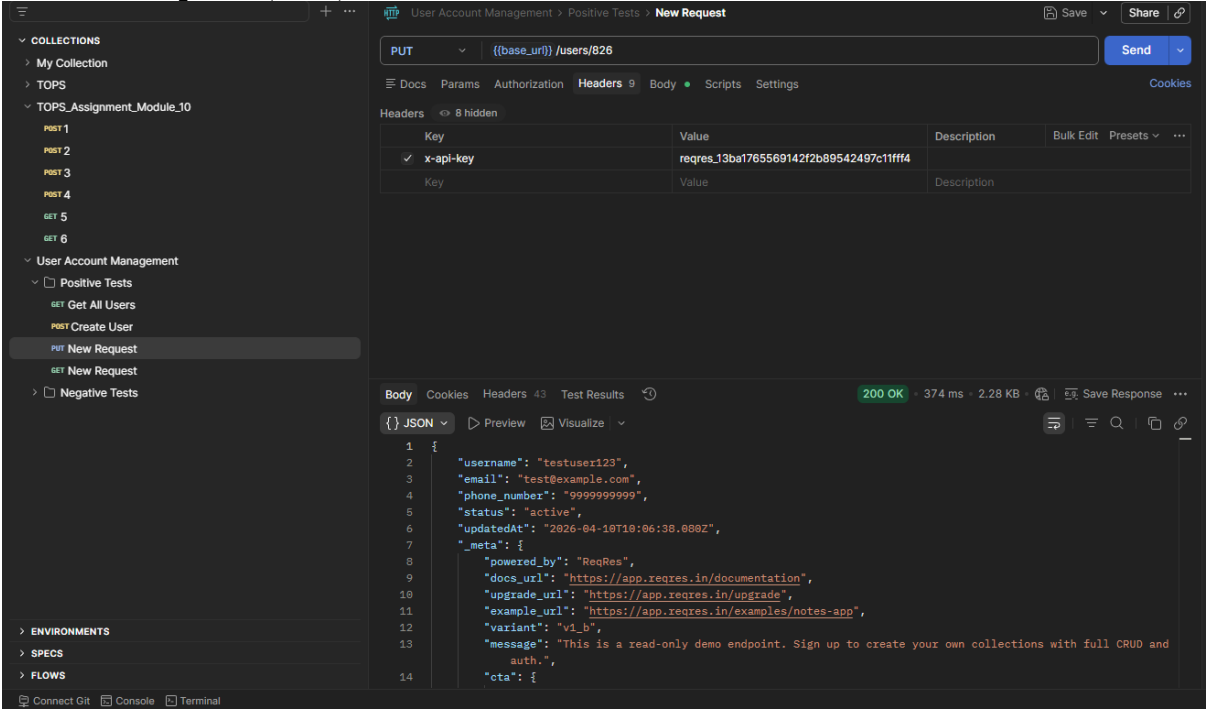




• 3. User Creation (POST):

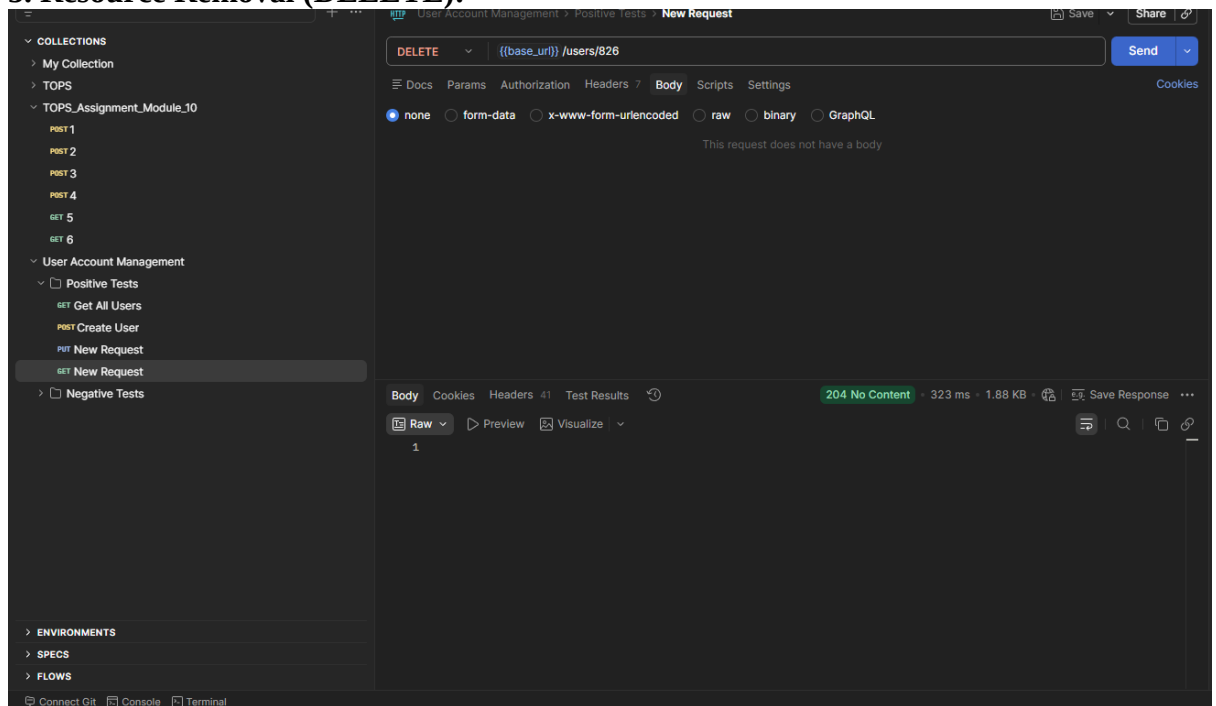


• 4. Account Update (PUT):



=====

- **5. Resource Removal (DELETE):**



Section C: Mini Capstone Project

1. Test Planning Document:

- **Scope:** Manual validation of the User Account Management APIs, specifically covering CRUD (Create, Read, Update, Delete) operations.
- **Objectives:** To manually verify that the API endpoints return the correct HTTP status codes and properly formatted JSON data.
- **Test Data Strategy:** Hardcoded JSON payloads entered manually into Postman's request body.
- **Authentication Approach:** Manual injection of API keys (e.g., x-api-key) into the Postman Headers tab when unauthorized errors occur.
- **Risks:** Using a public mock API (like Reqres) means data is not permanently saved, requiring workarounds for verifying deleted data.

- **Validation Techniques:** Purely manual visual inspection of the Postman response body, status codes, and headers.

2. Requirements Traceability Matrix:

Requirement ID	Requirement Description	HTTP Method	Endpoint Path	Manual Test Case ID
REQ-01	System must allow retrieving user data.	GET	/users/{id}	TC_01_RetrieveUser
REQ-02	System must allow creating a new user.	POST	/users	TC_02_CreateUser
REQ-03	System must allow updating user details.	PUT	/users/{id}	TC_03_UpdateUser
REQ-04	System must allow deleting a user.	DELETE	/users/{id}	TC_04_DeleteUser

3. End-to-End Test Scenario:

Scenario Name: User Account Lifecycle Validation

- **Step 1 (User Creation):** Manually send a POST request to /users with a JSON body. Visually verify the 201 Created status and note the generated id.
- **Step 2 (User Retrieval):** Manually send a GET request to /users/{id} using the newly generated ID. Visually verify the 200 OK status and ensure the user details match what was created.
- **Step 3 (User Update):** Manually send a PUT request to /users/{id} with a modified JSON body (e.g., changing the user's status). Visually verify the 200 OK status and check the response for the updated values.
- **Step 4 (User Deletion):** Manually send a DELETE request to /users/{id}. Visually verify the 204 No Content status, proving successful removal.

4. Test Summary Report:

=====

- **Executed Requests:** 4 manual End-to-End requests executed successfully via Postman.
- **Response Validation Results:** All API endpoints returned the expected HTTP status codes (200, 201, 204). JSON response payloads successfully passed manual visual inspection for structure and mandatory fields.
- **Defects Identified:** No functional defects identified in the core endpoints.
- **Release Recommendation:** The User Account Management APIs are stable, behave as expected under manual testing, and are **Recommended for Release**.